

面向室内复杂环境的智能巡检小车系统使用手册

项目使用手册 · 面向室内复杂环境巡检

版本 V1.0 · 2026-07-14

目录

1. 系统概述	5
2. 适用场景与典型任务	5
2.1 首次部署建图	5
2.2 日常自动巡检	6
2.3 动态障碍环境下的安全移动	6
2.4 识别通行标识并执行规则	6
2.5 人机协同巡检	7
2.6 环境安全数据采集	7
2.7 自然语言语音控制	7
3. 系统组成与代码结构	8
4. 网络连接与访问方式	9
4.1 基本网络要求	9
4.2 Web 访问地址	9
4.3 SSH 登录小车	9
5. Web 服务启动、停止与自启动	10
5.1 systemd 服务	10
5.2 手动启动方式	10
5.3 端口检查	11
5.4 日志查看	11
6. Web 控制台使用说明	11
6.1 页面入口	11
6.2 底盘启动	11
6.3 手动运动控制	12
6.4 摇杆控制	12

6.5 线速度和角速度	13
6.6 安全停止机制	13
7. 摄像头与远程画面	13
7.1 启动摄像头	13
7.2 摄像头视频流	14
7.3 常见问题	14
8. SLAM 建图与地图管理	14
8.1 启动 SLAM	14
8.2 RViz 可视化	15
8.3 保存地图	15
8.4 查看地图列表	15
8.5 选择地图	15
9. 导航与路径规划	16
9.1 启动导航	16
9.2 设置初始位置	16
9.3 设置目标点	16
9.4 路径显示	17
9.5 停止导航与清理	17
10. 自动避障与安全机制	17
10.1 障碍检测	17
10.2 自动停止	17
10.3 传感器融合	18
10.4 网络断连保护	18
11. YOLO 标识识别	18
11.1 数据集与训练	18
11.2 启动识别控制	18
11.3 功能逻辑	19
12. 人员识别与自动跟随	19
12.1 功能说明	19
12.2 使用方法	19
13. 多传感器环境数据模块	20
13.1 连接参数	20
13.2 传感器数据示例	20
13.3 Web 读取接口	20
13.4 字段说明	21

13.5 WiFi 注意事项	21
14. 大语言模型 Agent 语音控制	21
14.1 功能说明	21
14.2 支持的自然语言示例	22
14.3 单步动作示例	22
14.4 多步动作示例	22
14.5 后方方向修正规则	22
14.6 长距离运动	23
14.7 dry_run 测试	23
15. ROS2 分布式通信与双容器协同	24
15.1 ROS2 控制链路	24
15.2 ROS2 环境	24
15.3 关键 ROS2 topic	24
15.4 双容器协同	24
16. BJTU 扩展功能脚本	25
17. CI/CD 与工程质量保障	25
17.1 GitHub Actions	25
17.2 本地测试命令	26
17.3 工程意义	26
18. 常见问题与排查方法	26
18.1 Web 打不开	26
18.2 'jetson-desktop.local' 无法解析	27
18.3 HTTPS 证书不安全	27
18.4 摄像头黑屏	27
18.5 电压或电量显示为 0	27
18.6 运动按键无响应	28
18.7 语音失败：network	28
18.8 传感器无数据	29
19. 安全注意事项	29
20. 附录：主要接口与命令速查	29
20.1 Web 地址	29
20.2 服务命令	29
20.3 常用 API	29
20.4 传感器参数	30
20.5 语音 Agent dry_run 示例	30

20.6 ROS2 脚本30

20.7 训练脚本31

20.8 关键文件31

结语31

版本：V1.0

日期：2026-07-14

项目位置：</Users/lisvu/smartCar/BJTU-smartcar>

小车 Web 服务目录：/home/jetson/code/yahboomcar_ws/smartcar_web

1. 系统概述

本系统是一套面向室内复杂环境的智能巡检小车系统，目标场景包括高校实验室、机房、智能制造实训室、走廊、仓储空间等需要定期巡查的室内环境。

系统不是单一的遥控小车，而是围绕“巡检任务闭环”设计的一体化机器人系统。它能够完成环境建图、地图保存、导航规划、自动避障、摄像头画面回传、YOLO 标识识别、人员跟随、多传感器环境数据采集、Web 端控制、大语言模型语音控制和 CI/CD 工程保障。

系统整体能力可以概括为：

- 通过雷达进行 SLAM 建图，让小车认识室内环境。
- 通过 Nav2 在已保存地图上规划路径并导航。
- 通过雷达和传感器融合判断障碍风险，避免碰撞。
- 通过 YOLO 识别停止、左转、右转、前进、禁止通行等标识。
- 通过 YOLO 检测最近的人类目标，实现跟随。
- 通过传感器模块获取温度、湿度、光照、气压、烟雾、电量等数据。
- 通过 Web 控制台统一管理底盘、摄像头、SLAM、导航、传感器和语音控制。
- 通过 DeepSeek 大语言模型将自然语言语音命令拆解为小车可执行的运动指令序列。
- 通过 ROS2 分布式通信和容器化节点协同控制小车。
- 通过 GitHub Actions CI/CD 对关键代码进行自动测试。

2. 适用场景与典型任务

2.1 首次部署建图

当小车第一次进入实验室、机房或走廊时，它不知道周围空间结构。此时用户需要启动 SLAM 建图，让小车通过雷达扫描环境并生成地图。建图完成后，用户可以保存地图并命名，例如“实验室一层”“机房通道”“走廊区域”。

对应功能：

- 在线 SLAM 建图
- RViz 可视化集成
- 地图保存
- 地图命名
- 多地图管理

2.2 日常自动巡检

在日常巡检中，用户可以选择已经保存好的地图，设置小车初始位置和目标位置，小车自动规划路线并导航到目标区域。用户不需要每次手动遥控。

对应功能：

- 地图选择
- 初始位置设置
- 目标位置设置
- Nav2 路径规划
- 路径显示
- 停止导航与状态清理

2.3 动态障碍环境下的安全移动

室内环境中可能临时出现人员、桌椅、纸箱、设备等障碍物。小车在执行运动命令或导航任务时，需要实时判断前方风险，必要时停止或调整方向。

对应功能：

- 雷达障碍检测
- 障碍自动停止
- 雷达与传感器融合判断
- 自动调整方向
- 断连停止机制

2.4 识别通行标识并执行规则

在巡检区域中可能存在停止、左转、右转、前进、禁止通行等标识。小车通过摄像头采集图像并使用 YOLO 识别标识，随后将识别结果转换为运动策略。

对应功能：

- 自建数据集
- YOLO 微调重训练
- 五类标识识别
- 标识驱动运动控制

2.5 人机协同巡检

当巡检人员希望小车跟随自己移动时，小车可以使用摄像头检测画面中最近的人，并保持跟随，实现人机协同巡检。

对应功能：

- 人体检测
- 最近人员判断
- 自动跟随
- 摄像头画面回传

2.6 环境安全数据采集

实验室和机房巡检不仅要看通道是否畅通，还需要关注温湿度、光照、气压、烟雾等环境安全数据。系统通过传感器模块实时读取数据，并在 Web 页面显示。

对应功能：

- 传感器 WiFi 烧录
- TCP 传感器数据读取
- 温湿度、光照、气压、烟雾、电量显示

2.7 自然语言语音控制

普通用户不一定会 ROS 命令，也不想记复杂按钮。系统支持用户通过 Web 端语音输入自然语言，例如“前进十米”“去右前方四十五度零点三米处”“后退到左后方三十度零点二米处”。Web 端将语音转文字后交给 DeepSeek Agent 解析，后端将结果拆解成具体动作序列并执行。

对应功能：

- Web 端语音输入
- 语音转文字
- DeepSeek 指令理解

- 多步运动指令拆解
- 几何规则修正
- 长距离运动支持

3. 系统组成与代码结构

项目本地仓库位于：

/Users/lisvu/smartCar/BJTU-smartcar

主要目录如下：

```
BJTU-smartcar/
├── README.md
├── docs/
│   ├── CICD_GUIDE.md
│   └── WEB_CONTROL_STATUS.md
├── ROS2-Foxy/
│   └── scripts/
├── Rosmaster-App/
│   └── rosmaster/
├── yahboomcar_ws/
│   ├── smartcar_web/
│   ├── battery_monitor.py
│   ├── battery_status.py
│   ├── voice_cmd_vel.py
│   └── test/
├── yolo-train/
│   ├── train.sh
│   ├── setup_server.sh
│   └── 交通标志YOLO训练任务.md
└── wechat-miniprogram/
```

核心说明：

- [yahboomcar_ws/smartcar_web/](#)：Web 控制台源码备份，包括后端 [server.py](#)、前端页面和样式。
- [ROS2-Foxy/scripts/](#)：ROS2 分布式启动、桥接、控制和标识识别脚本。
- [Rosmaster-App/rosmaster/](#)：原始 Rosmaster 控制相关脚本和基础功能复现代码。
- [yolo-train/](#)：YOLO 数据集训练、服务器环境配置和训练脚本。
- [docs/](#)：CI/CD、Web 控制状态等文档。
- [wechat-miniprogram/](#)：曾经用于小程序测试的代码，目前主线功能集中在 Web 端。

小车端 Web 服务实际运行目录：

/home/jetson/code/yahboomcar_ws/smartcar_web

Web 服务容器内或 ROS mount namespace 中的运行目录可能为：

4. 网络连接与访问方式

4.1 基本网络要求

电脑、手机和小车必须处于同一局域网，否则无法访问 Web 端。此前常用网络为手机热点，小车 IP 示例为：

192.168.43.84

当电脑不在同一网段时，会出现：

jetson-desktop.local 解析失败
192.168.43.84 ping 不通
Web 页面无法打开

此时需要重新连接到小车所在热点或确认小车新的 IP 地址。

4.2 Web 访问地址

HTTP 地址：

http://192.168.43.84:8000

HTTPS 地址：

https://192.168.43.84:8443

如果 mDNS 可用，也可以访问：

https://jetson-desktop.local:8443

由于使用自签名证书，浏览器可能提示证书不安全。选择继续访问即可。

4.3 SSH 登录小车

小车默认 SSH 登录方式：

ssh jetson@jetson-desktop.local

如果主机名不可用，使用 IP：

ssh jetson@192.168.43.84

默认密码：

yahboom

5. Web 服务启动、停止与自启动

5.1 systemd 服务

Web 服务已经配置为系统服务：

smartcar-web.service

查看状态：

```
sudo systemctl status smartcar-web.service
```

启动：

```
sudo systemctl start smartcar-web.service
```

重启：

```
sudo systemctl restart smartcar-web.service
```

停止：

```
sudo systemctl stop smartcar-web.service
```

设置开机自启动：

```
sudo systemctl enable smartcar-web.service
```

5.2 手动启动方式

进入 Web 服务目录：

```
cd /home/jetson/code/yahboomcar_ws/smartcar_web
```

手动运行：

```
python3 server.py
```

启动后输出类似：

```
Smart car web console: http://0.0.0.0:8000  
Smart car web console: https://0.0.0.0:8443
```

5.3 端口检查

检查 Web 服务端口：

```
ss -lntp | grep -E ':(8000|8443)'
```

正常情况应看到：

```
0.0.0.0:8000  
0.0.0.0:8443
```

5.4 日志查看

查看 Web 服务日志：

```
journalctl -u smartcar-web.service -n 100 --no-pager
```

持续查看：

```
journalctl -u smartcar-web.service -f
```

6. Web 控制台使用说明

6.1 页面入口

Web 控制台入口：

<https://192.168.43.84:8443>

主要页面包括：

- 控制台：小车运动控制、摇杆控制、摄像头画面、语音控制。
- SLAM 页面：建图、RViz 可视化、地图保存。
- 导航页面：地图选择、初始位置、目标位置、路径规划。
- 传感器页面：温湿度、光照、气压、烟雾、电量等数据。
- 电压页面：电压、电量或底盘状态显示。
- BJTU 扩展页面：扩展脚本、检测、融合、标识识别等功能。

6.2 底盘启动

在 Web 控制台中点击“启动底盘”或相关按钮，后端会通过 [ProcessManager](#) 启动底盘/雷达相关 ROS2 节点。

对应后端 API：

POST /api/process/start

示例请求：

```
{  
  "name": "bringup"  
}
```

6.3 手动运动控制

支持动作：

- 前进
- 后退
- 左转
- 右转
- 左平移
- 右平移
- 停止

后端运动接口：

POST /api/move
POST /api/stop

[/api/move](#) 示例：

```
{  
  "linear_x": 0.12,  
  "linear_y": 0.0,  
  "angular_z": 0.0  
}
```

后端会发布 ROS2 [/cmd_vel](#)：

geometry_msgs/msg/Twist

6.4 摇杆控制

Web 控制台设计了圆形摇杆。拖动摇杆可以连续控制小车方向：

- 向上拖动：前进

- 向下拖动：后退
- 向左拖动：左转
- 向右拖动：右转
- 左右平移通过独立控制方式完成

摇杆适用于精细控制，特别是在靠近目标区域、避开桌椅、调整摄像头视角时使用。

6.5 线速度和角速度

Web 端支持调整线速度和角速度。建议：

- 室内狭窄区域使用低速。
- 开阔区域可以适当提高速度。
- 调试 SLAM 或导航时优先使用稳定低速。

6.6 安全停止机制

系统设计为：小车只有持续接收到运动命令才会持续运动。如果 Web 连接断开或不再发送运动命令，小车会停止，避免网络异常导致持续前进。

手动停止：

POST /api/stop

语音停止：

停止
停车
急停

7. 摄像头与远程画面

7.1 启动摄像头

Web 端点击“启动相机”后，后端会重启摄像头相关进程，并重置摄像头读取状态。

对应 API：

POST /api/process/start

请求示例：

{

```
"name": "camera"  
}
```

7.2 摄像头视频流

摄像头流接口：

GET /api/camera/stream

当前实现方式：

- 后端使用 OpenCV 直接读取 `/dev/video0`。
- 输出 MJPEG 流。
- 不依赖 ROS Image topic。

7.3 常见问题

如果摄像头画面黑屏：

1. 检查摄像头是否插好。
2. 点击 Web 端“启动相机”。
3. 检查 `/dev/video0` 是否存在。
4. 查看 Web 服务日志。
5. 尝试重启 Web 服务。

8. SLAM 建图与地图管理

8.1 启动 SLAM

Web 端点击启动建图后，系统会启动底盘/雷达和 SLAM 相关节点。

常用进程名：

```
bringup  
slam  
mapping_keyboard  
save_map
```

相关 API：

```
POST /api/mapping/start  
POST /api/process/start
```

8.2 RViz 可视化

Web 端集成 RViz 或可视化页面，用于显示：

- 地图
- 雷达点
- 机器人位姿
- 初始位置
- 目标位置
- 规划路径

RViz 中可以使用原生工具设置初始位置和目标点。

8.3 保存地图

建图完成后，在 Web 端点击保存地图，并输入地图名称。

对应 API：

POST /api/maps/save

示例：

```
{  
  "name": "实验室一层"  
}
```

地图保存后会出现在地图列表中，后续导航可直接选择。

8.4 查看地图列表

接口：

GET /api/maps

返回内容包括：

- 已保存地图列表
- 当前选中的地图

8.5 选择地图

接口：

POST /api/maps/select

示例：

```
{  
  "map": "/root/maps/lab1.yaml"  
}
```

9. 导航与路径规划

9.1 启动导航

导航前需要：

1. 选择已保存地图。
2. 启动底盘/雷达。
3. 启动导航进程。
4. 设置初始位置。
5. 设置目标位置。

对应进程：

```
nav_dwa  
nav_teb
```

9.2 设置初始位置

接口：

POST /api/nav/initial_pose

示例：

```
{  
  "x": 0.0,  
  "y": 0.0,  
  "theta": 0.0  
}
```

9.3 设置目标点

接口：

POST /api/nav/goal

示例：

```
{  
  "x": 1.2,  
  "y": 0.6,  
  "theta": 0.0  
}
```

9.4 路径显示

后端订阅 `/plan`，前端在地图上绘制路径。可视化接口：

GET `/api/viz`

返回内容包括：

- `map`
- `scan`
- `pose`
- `plan`
- `ages`

9.5 停止导航与清理

停止导航后应清理上一次初始位置、目标点和路径显示。

接口：

POST `/api/nav/clear`

10. 自动避障与安全机制

10.1 障碍检测

系统使用雷达检测周围障碍物距离。Web 状态接口会返回障碍距离相关信息：

GET `/api/status`

10.2 自动停止

当小车检测到前方距离低于安全阈值时，会阻止继续前进或执行停止。

相关接口：

POST /api/guard

可配置内容包括：

- 是否开启避障
- 停止距离

10.3 传感器融合

自动避障不只依赖单一运动命令，还结合雷达、传感器、运动状态等信息综合判断。其目标是避免小车在执行长距离语音命令或导航命令时撞上临时障碍。

10.4 网络断连保护

系统设计为只有持续收到运动命令才继续运动。如果 Web 端断开或命令停止，小车会自动停止。

11. YOLO 标识识别

11.1 数据集与训练

YOLO 训练代码位于：

yolo-train/

主要文件：

train.sh
setup_server.sh
交通标志YOLO训练任务.md

识别类别：

- 停止
- 左转
- 右转
- 前进
- 禁止通行

11.2 启动识别控制

启动脚本：

ROS2-Foxy/scripts/start_sign_control.sh

停止脚本：

ROS2-Foxy/scripts/stop_sign_control.sh

相关节点脚本：

ROS2-Foxy/scripts/sign_command_node.py

11.3 功能逻辑

摄像头捕获画面后，YOLO 模型识别标识类别。系统根据识别结果执行对应运动策略：

- 停止：小车停止。
- 左转：小车左转。
- 右转：小车右转。
- 前进：小车前进。
- 禁止通行：小车停止或避免进入该区域。

12. 人员识别与自动跟随

12.1 功能说明

人员跟随功能使用视觉模型检测画面中的人，并选择最近的人类目标作为跟随对象。

适用场景：

- 巡检人员带领小车移动。
- 小车跟随工作人员采集环境画面。
- 展示人机协同巡检能力。

12.2 使用方法

1. 启动摄像头。
2. 启动人员检测/跟随相关节点。
3. 确保画面中有清晰的人体目标。
4. 小车会根据目标位置调整运动。

注意：跟随功能应在空旷区域测试，避免近距离碰撞。

13. 多传感器环境数据模块

13.1 连接参数

传感器模块默认 TCP 参数：

Host: 192.168.1.11
Port: 8888

协议类型：

TCP 长连接

上行数据格式：

UTF-8 JSON + \r\n

13.2 传感器数据示例

```
{
  "services": [
    {
      "service_id": "sensorData",
      "properties": {
        "temperature": "25.30",
        "humidity": "58.10",
        "illumination": "30.00",
        "smoke": "2.00",
        "pressure": "101.10",
        "longitude": "119.090000",
        "latitude": "36.680000",
        "battery": "98"
      }
    }
  ]
}
```

13.3 Web 读取接口

接口：

GET /api/sensors

默认使用：

192.168.1.11:8888

也可以通过 query 参数指定：

```
/api/sensors?host=192.168.1.11&port=8888
```

13.4 字段说明

字段	含义	单位/说明
temperature	温度	摄氏度
humidity	湿度	%RH
illumination	光照	相对值
smoke	烟雾/可燃气体	相对值
pressure	大气压	hPa
longitude	经度	十进制度
latitude	纬度	十进制度
battery	电量	百分比

13.5 WiFi 注意事项

传感器模块需要与电脑或小车处于可互通网络。此前使用过两种方式：

1. 电脑直接连接传感器热点，访问 192.168.1.11:8888。
2. 将传感器 WiFi 烧录到手机热点，使电脑、小车、传感器处于同一网络。

如果 Web 能打开但传感器没有数据，优先检查：

- 电脑/小车是否能 ping 通 192.168.1.11。
- TCP 端口 [8888](http://192.168.1.11:8888) 是否可连接。
- 传感器模块是否已接入当前热点。

14. 大语言模型 Agent 语音控制

14.1 功能说明

Web 端支持语音输入。浏览器将语音转成文字后，请求后端：

```
POST /api/agent/voice
```

后端调用 DeepSeek 大语言模型进行自然语言理解，并将文本命令转换成小车可执行动作。

14.2 支持的自然语言示例

前进十米
后退一米
停止
启动底盘
打开摄像头
启动建图
启动导航
去右前方四十五度零点三米处
后退到左后方三十度零点二米处

14.3 单步动作示例

用户说：

前进十米

Agent 输出：

```
{  
  "action": "move_forward",  
  "distance_m": 10.0,  
  "message": "前进10米"  
}
```

14.4 多步动作示例

用户说：

前进到右前方四十五度零点三米处

Agent 输出：

```
{  
  "action": "sequence",  
  "steps": [  
    {"action": "move_forward", "distance_m": 0.212},  
    {"action": "turn_right", "angle_deg": 90},  
    {"action": "move_forward", "distance_m": 0.212}  
  ]  
}
```

14.5 后方方向修正规则

为了避免大模型简单将“左”理解为左转，后端加入几何运动规则修正。

规则如下：

语义	动作序列
左前方	前进 -> 左转 -> 前进
右前方	前进 -> 右转 -> 前进
左后方	后退 -> 右转 -> 后退
右后方	后退 -> 左转 -> 后退

例如用户说：

后退到左后方三十度零点二米处

系统输出：

```
[
  {"action": "move_backward", "distance_m": 0.173},
  {"action": "turn_right", "angle_deg": 90},
  {"action": "move_backward", "distance_m": 0.1}
]
```

14.6 长距离运动

当前已经取消 1.5 米距离限制。用户说：

前进十米

后端会解析为：

```
{
  "distance_m": 10.0
}
```

执行时间按距离和速度计算。默认线速度约为 0.12 m/s，因此 10 米大约需要 83 秒。执行过程中可以随时使用停止命令。

14.7 dry_run 测试

如果只想测试语音解析，不让小车移动，可以使用：

```
{
  "text": "前进十米",
  "dry_run": true
}
```

接口：

POST /api/agent/voice

15. ROS2 分布式通信与双容器协同

15.1 ROS2 控制链路

核心运动链路：

```
Web 按钮 / 语音 Agent
-> /api/move 或 /api/agent/voice
-> smartcar_web 后端
-> /cmd_vel
-> driver_node
-> 底盘执行
```

15.2 ROS2 环境

Web 后端启动 ROS 进程时使用的环境包括：

```
source /opt/ros/foxy/setup.bash
source /root/icar_ros2_ws/icar_ws/install/setup.bash
export ROS_DOMAIN_ID=32
export ROS_LOCALHOST_ONLY=0
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
export ROBOT_TYPE=x3
export RPLIDAR_TYPE=a1
```

15.3 关键 ROS2 topic

Topic	作用
/cmd_vel	小车速度控制
/scan	雷达数据
/map	SLAM 地图
/odom	里程计
/plan	导航路径
/voltage	电压状态

15.4 双容器协同

系统通过容器中的 ROS2 节点协同工作。[ROS2-Foxy/scripts/start_all.sh](#) 用于启动分布式通信、Jetson 侧驱动和本机侧桥接。

主要脚本：

```
ROS2-Foxy/scripts/start_all.sh
```



```
ROS2-Foxy/scripts/stop_all.sh
ROS2-Foxy/scripts/run_bridge_mac.sh
ROS2-Foxy/scripts/run_bridge_jetson.sh
ROS2-Foxy/scripts/run_drive_bridge_mac.sh
ROS2-Foxy/scripts/run_drive_bridge_jetson.sh
ROS2-Foxy/scripts/run_base_driver_jetson.sh
```

16. BJTU 扩展功能脚本

Web 后端包含 BJTU 扩展功能管理能力，接口包括：

```
GET /api/bjtu/status
GET /api/bjtu/log
GET /api/bjtu/detections
POST /api/bjtu/start
POST /api/bjtu/stop
```

本地扩展脚本包括：

```
ROS2-Foxy/scripts/fusion_node.py
ROS2-Foxy/scripts/sign_command_node.py
ROS2-Foxy/scripts/detect_headless.py
ROS2-Foxy/scripts/start_sign_control.sh
ROS2-Foxy/scripts/stop_sign_control.sh
```

扩展能力包括：

- 静态融合
- 标识识别控制
- YOLO 检测
- 检测日志查看
- 扩展功能启动和停止

17. CI/CD 与工程质量保障

17.1 GitHub Actions

仓库配置了 GitHub Actions，对推送和 Pull Request 进行自动检查。

主要检查：

- Python 语法检查
- 单元测试
- 覆盖率检查

- ROS2 Foxy 容器内 `bjtu_comm` 构建与测试

17.2 本地测试命令

ROS2 检查：

```
source /opt/ros/foxy/setup.bash
cd bjtu_ros2_ws
colcon build --packages-select bjtu_comm
colcon test --packages-select bjtu_comm
colcon test-result --verbose
```

Python 单元测试：

```
python3 -m pip install coverage pytest
export PYTHONPATH="$PWD/bjtu_ros2_ws/src/bjtu_comm:$PWD/yahboomcar_ws"
python3 -m coverage run \
  --source=bjtu_comm.messages,battery_status \
  -m pytest \
  bjtu_ros2_ws/src/bjtu_comm/test/test_messages.py \
  yahboomcar_ws/test/test_battery_status.py
python3 -m coverage report --show-missing --fail-under=100
```

17.3 工程意义

CI/CD 用于防止新功能破坏已有能力。由于系统涉及 Web、ROS2、YOLO、传感器和语音 Agent，多模块协作复杂，自动测试可以提高迭代可靠性。

18. 常见问题与排查方法

18.1 Web 打不开

检查步骤：

1. 确认电脑和小车在同一网络。
2. ping 小车 IP：

```
ping 192.168.43.84
```

3. 检查端口：

```
curl -k https://192.168.43.84:8443/api/status
```

4. SSH 到小车检查服务：

```
sudo systemctl status smartcar-web.service
```

5. 检查端口监听：

```
ss -lntp | grep -E ':(8000|8443)'
```

18.2 jetson-desktop.local 无法解析

使用 IP 访问：

```
https://192.168.43.84:8443
```

或者重新连接同一热点后再试。

18.3 HTTPS 证书不安全

当前使用自签名证书，浏览器会提示不安全。选择继续访问即可。

18.4 摄像头黑屏

检查：

```
ls /dev/video*
```

然后在 Web 端点击“启动相机”，或重启服务：

```
sudo systemctl restart smartcar-web.service
```

18.5 电压或电量显示为 0

可能原因：

- 底盘未启动。
- `/voltage` topic 未发布。
- Web 后端未连接 ROS 图。
- 电量来源切换到传感器模块但传感器不可达。

处理：

1. 点击“启动底盘”。
2. 检查 `/api/status`。
3. 检查 ROS topic。
4. 检查传感器 TCP 连接。

18.6 运动按键无响应

可能原因：

- 底盘未启动。
- `/cmd_vel` 被其他节点抢占。
- ROS_DOMAIN_ID 不一致。
- 底盘驱动或串口被其他进程占用。
- 小车和电脑不在同一网络。

处理：

1. 点击“启动底盘”。
2. 停止语音 Demo、视觉控制、手柄控制等可能抢占 `/cmd_vel` 的进程。
3. 查看 Web 日志。
4. 重启 `smartcar-web.service`。

18.7 语音失败：network

常见原因：

- 手机/浏览器没有通过 HTTPS 访问。
- 浏览器不允许录音权限。
- 手机和小车不在同一网络。
- Web 后端 `/api/agent/voice` 不可达。
- DeepSeek API 网络请求失败。

处理：

1. 使用 HTTPS 地址打开：

`https://192.168.43.84:8443`

2. 允许浏览器麦克风权限。
3. 测试接口：

```
curl -k -X POST https://192.168.43.84:8443/api/agent/voice \  
-H "Content-Type: application/json" \  
--data '{"text":"状态","dry_run":true}'
```

4. 查看 Web 服务日志。

18.8 传感器无数据

检查：

```
nc 192.168.1.11 8888
```

正常会返回 JSON 数据。如果连不上，检查传感器 WiFi 是否接入当前热点。

19. 安全注意事项

- 1. 第一次测试运动命令时，应将小车放在开阔区域。
- 2. 测试长距离语音命令前，应确认自动避障已开启。
- 3. 语音命令如“前进十米”会持续较长时间，必须确保可以随时停止。
- 4. 测试人员跟随时，应保持人与小车距离，避免急停不及时。
- 5. 不要同时启动多个会发布 `/cmd_vel` 的控制节点。
- 6. 不要在楼梯、桌边、狭窄边缘测试自动导航。
- 7. Web 断开时小车应自动停止，但仍建议现场有人看护。
- 8. 修改 DeepSeek API Key 时不要提交到 GitHub。

20. 附录：主要接口与命令速查

20.1 Web 地址

```
http://192.168.43.84:8000
https://192.168.43.84:8443
```

20.2 服务命令

```
sudo systemctl status smartcar-web.service
sudo systemctl restart smartcar-web.service
journalctl -u smartcar-web.service -n 100 --no-pager
```

20.3 常用 API

方法	路径	作用
GET	<code>/api/status</code>	获取小车状态
GET	<code>/api/viz</code>	获取地图、雷达、位姿、路径
GET	<code>/api/maps</code>	获取地图列表

方法	路径	作用
GET	/api/sensors	获取传感器数据
GET	/api/camera/stream	摄像头 MJPEG
POST	/api/move	发布运动速度
POST	/api/stop	停止小车
POST	/api/guard	配置避障
POST	/api/process/start	启动进程
POST	/api/process/stop	停止进程
POST	/api/mapping/start	启动建图
POST	/api/maps/save	保存地图
POST	/api/maps/select	选择地图
POST	/api/nav/initial_pose	设置初始位置
POST	/api/nav/goal	设置目标点
POST	/api/nav/clear	清理导航状态
POST	/api/agent/voice	语音 Agent 控制
GET	/api/bjtu/status	BJTU 扩展功能状态
POST	/api/bjtu/start	启动 BJTU 扩展功能
POST	/api/bjtu/stop	停止 BJTU 扩展功能

20.4 传感器参数

Host: 192.168.1.11
Port: 8888

20.5 语音 Agent dry_run 示例

```
curl -k -X POST https://192.168.43.84:8443/api/agent/voice \
-H "Content-Type: application/json" \
--data '{"text":"后退到左后方三十度零点二米处","dry_run":true}'
```

20.6 ROS2 脚本

```
ROS2-Foxy/scripts/start_all.sh
ROS2-Foxy/scripts/stop_all.sh
ROS2-Foxy/scripts/start_sign_control.sh
ROS2-Foxy/scripts/stop_sign_control.sh
ROS2-Foxy/scripts/run_base_driver_jetson.sh
```

```
ROS2-Foxy/scripts/run_bridge_mac.sh
ROS2-Foxy/scripts/run_bridge_jetson.sh
```

20.7 训练脚本

```
yolo-train/setup_server.sh
yolo-train/train.sh
```

20.8 关键文件

```
smartcar_web/server.py
smartcar_web/static/index.html
smartcar_web/static/app.js
smartcar_web/static/sensors.html
smartcar_web/static/slam.html
smartcar_web/static/detail.js
```

结语

本系统围绕室内复杂环境巡检需求，将 Web 控制、SLAM 建图、导航规划、自动避障、YOLO 视觉识别、人员跟随、多传感器环境采集、大语言模型语音控制、ROS2 分布式通信和 CI/CD 工程保障集成到同一平台中，使小车从基础遥控平台升级为具备自主巡检与自然语言交互能力的智能机器人系统。